Content mapping — where 7 contents and 8 questions land

Kidsverse+ — from the generated package to the three lesson screens, step by step

For **learning engine + product** Date **8 September 2026** Companion to **Content Slots** and **Content Contract**

> **The mapping in one sentence**
>
> The engine returns **7 teaching contents** and **8 questions** for one image. The app has **three templates**: a teaching screen, a check screen, an outcome screen. The mission is played as a **sequence of teach→check pairs**: each content fills the teaching template, the question that belongs to it fills the check template, and the outcome screen is computed from which checks were passed. The link between a content and its question is one field: `related_content_id` .

1. The 7 contents → the Discover template (screen 14)

Every content, whatever its type, fills the **same** slots. What changes per content is the words, the picture and which representation is the default. The engine's own examples, mapped:

#	Engine content	Example (from the engine spec)	Model / picture	Fills on screen 14
1	Introduction	*This is one whole pizza.*	pizza, `split:[1]` (whole)	model.title "1 whole pizza", prompt.statement = the sentence, prompt.question "What is this?"
2	Equal Sharing	*Four astronauts share one pizza equally.*	pizza `[1,1,1,1]` + crew of 4	the shipped mission: crew.count 4, statement, question "What do you notice?"
3	Fraction Representation	*Each equal part represents 1/4.*	pizza `[1,1,1,1]`, one part highlighted	think_about = the sentence; hints walk from "count the parts" to "one of four"
4	Numerator	*1 tells us how many parts we have.*	bar `[1,1,1,1]`, one shaded	model.caption; easier = 2 parts, 1 shaded
5	Denominator	*4 tells us the total equal parts.*	number line, 4 jumps	model.caption; hints count the jumps
6	Compare Parts	*Are all four parts the same size?*	pizza with an uneven cut	prompt.question = the sentence — and this content's check is the shipped Spot the Mistake
7	Real-world Application	*4 friends share equally; each gets 1/4.*	bar, crew of 4 (friends)	statement + question "How much does each friend get?"

The teaching template never changes. A content that does not fit these slots is a content that should be rewritten, not a template that should grow.

2. The 8 questions → which template

Questions are not all one shape. One of the eight runs on the check template today. Six more run once that template takes 2–4 options instead of 2 — one change to one screen. One needs a widget the app does not have. Being honest about this is the point of this table.

#	Engine question	Example	Answer widget	Template	Status

1	Identification	*How many equal parts?*	pick one of 2–4 (3 / 4 / 5)	`check`	**after 2–4 options** options[] with numbers
2	Fraction	*What fraction represents one part?*	pick one of 4 fractions	`check`	**after 2–4 options** options[] = fraction chips; Test Arena already renders these
3	Concept	*What does each astronaut get?*	pick one of 3	`check`	**after 2–4 options** options[]
4	Denominator	*What does 4 in 1/4 mean?*	pick one of 3 sentences	`check`	**after 2–4 options** options[]; sub-line carries the sentence
5	Numerator	*What does 1 in 1/4 mean?*	pick one of 3 sentences	`check`	**after 2–4 options** as above
6	Visual YES/NO	*Are these parts equal?*	YES / NO	`check`	**runs today** **the shipped question**, verbatim
7	Scenario	*If another person joins, is it still 1/4 each?*	YES / NO + a one-line why	`check`	**needs new widget** yes/no works today; the "why" line needs a text field — not built
8	Application	*Which fraction shows 3 of 4 parts?*	pick one of 4 fractions, picture-led	`check`	**after 2–4 options** options[] = fractions; picture from `split`

One template, generalised — not eight

Every question above is "show a picture, offer 2–4 options, one is right". The check screen already does that for two options. Letting it take up to four, with fraction chips as labels, covers 7 of 8. That is one change to one screen. Only the scenario's free-text "why" is genuinely new.

3. The sequence — how a child moves through it

Not "7 screens then 8 screens". Each teach is followed by its own check, so a misunderstanding is caught where it happens. The engine links them; the client walks the list.

```
Discover(c1 Introduction)            → Check(q1 How many parts?)
Discover(c2 Equal sharing)           → Check(q3 What does each get?)
Discover(c3 Fraction representation) → Check(q2 What fraction is one part?)
Discover(c4 Numerator)               → Check(q5 What does 1 mean?)
Discover(c5 Denominator)             → Check(q4 What does 4 mean?)
Discover(c6 Compare parts)           → Check(q6 Are these equal?)      ← shipped today
Discover(c7 Real-world)              → Check(q7 Another person joins?) → Check(q8 3 of 4?)
                                                                  ↓
                                        Mission Complete (computed from the 8 results)
```

Rules the client applies

- **Order** = `content_no`. A question follows the content its `related_content_id` names. Two questions on one content run back to back (c7 above).
- **Wrong answer** → the check's own hints, then "Show it another way" (the same question on the next model). The child is not sent backwards; the miss is recorded.
- **Two misses on one content** → the client swaps the *teaching strategy*: it re-opens the same content with its alternate model as default. This is what the engine's "teaching strategy" becomes on screen — a second picture, not a second lesson.
- **Time budget**: 7 teach + 8 check is ~15 minutes. The mission card's `duration_min` must say so; today it says 6–8 because there is one pair.

4. Mission Complete — computed, not authored

On screen 17	Comes from
Mastery ring	correct checks ÷ 8, weighted down by hints and attempts (formula in the Data Model PDF)
"You understood" card	the `skill` whose checks were all passed first try
"You improved" card	the skill passed after a hint or a model switch
"Next" card	the skill with a miss, or the engine's `next_recommended_topic` if none
XP	`mission.xp` + each check's `xp_on_correct` actually earned

So the engine authors the three cards' *wording* per skill; the client decides *which* card each skill lands in. That is why `outcomes[]` carries a `skill` and each question carries a `skill` too.

5. What this adds to the package

Three fields. Everything else in the Content Contract stands.

```
contents[n].content_no          -- already there; now it is the play order
questions[n].related_content_id -- which content this check follows
questions[n].skill              -- "Equal parts" / "Numerator" ... for the outcome cards
questions[n].options[]          -- 2 to 4 now, not 2
```

6. Two decisions needed before the engine is built against this

1. **All 7, or a subset per session?** Seven pairs is a long sitting for a young child. The shape allows the engine to return all seven and the client to play, say, three per day and resume. Resuming needs the run to be saved — the same open question as in the Data Model PDF.
2. **Scenario question's free text**: build a text-answer widget, or have the engine phrase scenarios as pick-one. The second is zero frontend work and loses little.

> **What exists today vs. what this needs**
> **Exists:** the three templates, JSON-driven, with one content and one yes/no question playing end to end. **To build:** the check screen taking 2–4 options with fraction labels (small), the pair-walker that steps through the list (small), the outcome computation (medium, needs the event table). **From the engine:** the three fields above, and 7 + 8 real items for one topic to walk through together.
